POLICY DESK

Win Plan · Judge-Aligned Execution Brief

University Blockchain Conference 2026 Hackathon

Simulated judges based on plan + gallery notes — not a verified code audit or organizer judging.

Engineering team

Engineer	Discipline	Owns
Dipak Yadav (@dipak5501)	Full-stack developer · product/engineering lead	Task UX, APIs, demo, Devpost, merge
Utkarsh Lubal (@ubiiii)	Cybersecurity engineer	Threat model, adversarial tests, audit honesty
Kashif Manzer (@KashifManzer)	Blockchain engineer	Solana settle, intent binding, x402, Streams

No invented employers, years of experience, or certifications.

Submit: Fri Nov 20, 12:00 PM CST · Artifact: **policy-desk** only · Planning PDF — app code unchanged in this revision

Contents

- 1. Three-judge review summary + differentiation + 12-step workflow
- 2. Win plan (agreed research-agent direction)
- 3. Architecture (workflow-aligned)
- 4. Feature backlog (P0 rewritten to judges)
- 5. Team split (5-day / roles)
- 6. Threat model
- 7. Organizer resources + Streams
- 8. Appendix A — all organizer/sponsor URLs · Appendix B constants

1. Three-judge review

Disclaimer: *Simulated judges based on plan + gallery notes — **not** a verified code audit or organizer judging.*

Agreed headline: Strengthen **one complete agent workflow** before adding more features.

Review summary

Judge	Main objection	Agreed decision
Product and originality	“This could look too similar to Aegis or Agent Guard.”	Show an agent purchasing useful information, recovering from a blocked request, and completing its task.
Blockchain engineering	“A successful transfer alone does not prove a reliable payment system.”	Bind quotes to transactions, confirm settlement, and prevent duplicate payments during retries.
Security	“Security claims need demonstrable evidence.”	Make adversarial tests and visible rejection reasons essential deliverables.

Vault and sponsored fees were considered. **Decision: defer the vault; make sponsorship optional.** Useful agent workflow, transaction correctness, and security evidence come first.

Differentiation vs Aegis / Agent Guard

Gallery notes tagged **Aegis** as “limited overlap,” but its stated purpose (agent vaults, hard limits, human overrides) makes it a **close conceptual competitor**. Use that comparison to sharpen differentiation — do not dismiss it.

Competitor	What they emphasize	How Policy Desk must differ
Aegis (Solana)	Smart vaults, hard limits, one-click human overrides	We do **not** ship a new custody vault this weekend. We **block before sign**, enforce server policy + budgets, and complete a **research purchase workflow** with receipts tied to useful content.
Agent Guard (Base / Polymarket)	Policy engine, visible decisions, on-chain decision logs for trading	We are **Solana Devnet USDC + x402 agent commerce**, not a Polymarket sandbox. Borrow **visible rule pass/fail + reasons**; do not copy trading-market scope.

Differentiation one-liner: A research agent buys useful data within a budget, handles a blocked purchase safely, requests approval when needed, and returns an answer backed by purchased information and payment receipts.

Agreed product direction

One complete workflow (narrow research task + small approved resource catalog — not a marketplace):

A research agent buys useful data within a user-defined budget, handles a blocked purchase safely, requests approval when necessary, and returns an answer supported by purchased information and payment receipts.

12 demonstration steps

1. User starts a task with a defined USDC budget.
2. A real model uses constrained tools to inspect available resources *(fixture mode allowed if labeled; never silent script-as-live)*.
3. User inspects free preview, description, price, and merchant.
4. Agent requests a resource through the policy API.
5. Server evaluates deterministic rules.
6. A prohibited request is **blocked**.
7. Agent chooses an approved alternative and continues.
8. An above-threshold purchase pauses for **operator approval**.
9. Wallet authorizes the prepared transaction (Phantom).
10. Settlement is **confirmed** before paid content is delivered.

11. Agent uses purchased content in its final answer.
12. A **receipt** connects task, decision, payment, resource, and answer.

Team-operated merchant endpoint is acceptable if clearly labeled. Use a **separate merchant wallet** for the primary demo. Keep a **deterministic fixture mode** for reliable demos, visibly labeled.

Gallery features to adapt (priorities)

Inspiration	Feature for Policy Desk	Priority
Quarry	Resource preview → exact quote → payment → useful result	**Essential**
SeaPay	Approval card that pauses and resumes the agent's task	**Essential**
Agent Guard	Visible policy checks with concrete rejection reasons	**Essential**
Quarry / PolyHedge	Structured receipt connecting purchase to result	**Essential**
FRCT	Clear balances, reserved funds, and remaining budget	**Essential**
Gasless Hub	Sponsored transaction fees	**Optional** after core works

Security foundation (requirements)

• Policy enforcement stays on the **server**. Model must not control recipients, mint, limits, policy config, operator credentials, or arbitrary merchant/RPC URLs.
• Schema-validated **resource IDs** resolved through an approved server-side catalog.
• Bind payment intents to wallet, task, resource, merchant, network, mint, exact base-unit amount, quote expiry, and policy version.
• Integer **base units** for all financial math.
• **Atomic, durable budget reservation** (smallest storage appropriate). Concurrency model must be defined.
- Two concurrent requests cannot exceed remaining budget.
- Rejection / safe expiry releases reservations exactly once.
- Uncertain settlement stays reserved until reconciled.
- Approval applies only to its bound intent; changed/expired intents cannot reuse approval.
- Operator credentials unavailable to the agent; auth scoped to wallet/session.
• Validate the **complete transaction** against a permitted instruction template. Finding one matching `TransferChecked` is insufficient.
- Reject unexpected transfers, programs, delegate approvals, authority changes, account closures, fee payers, or excessive compute/fee instructions.
- Define exactly which legitimate setup instructions are permitted.
• **Idempotent retries.** Reconcile an existing transaction before creating a replacement. Timeout ≠ failure.
• Mark settlement successful only after verifying expected tx succeeded on configured network at documented commitment.
• Separate payment confirmation from resource delivery; if paid but delivery fails, retry fulfillment **without** charging again.
• Pause/policy changes effective at last execution boundary the app controls; document that the server cannot revoke an already signed tx broadcast elsewhere.

Signing and audit honesty

Distinguish clearly:

1. Policy authorization
2. Operator approval
3. Wallet signature
4. On-chain confirmation
5. Resource delivery

If Phantom signs every payment, under-threshold means **no additional operator approval** — not autonomous settlement.

Do not add EVM-style ERC-20 allowance approval to ordinary SPL transfers.

Describe server-controlled audit storage accurately: local append-only files + hashes alone are **not** an independently immutable on-chain audit trail.

Streams (conditional) · Vault · Sponsorship

Item	Decision
QuickNode Streams	After core works: reconcile known payments + flag unmatched outgoing transfers. Auth, dedupe, delay tracking, outcome verify. Must improve monitoring — not merely animate a ticker. Manual webhook ≠ proof of integration. Incomplete → remove bounty claim.
On-chain vault	**Deferred**
Sponsored fees	**Optional** after core; fee limits + abuse controls; disclose ATA creation cost coverage; never claim sponsorship removes wallet authorization

Tests + security demo evidence

Required tests: unauthorized merchant/resource; wrong mint/amount/recipient/authority/network; extra malicious instructions; missing/expired/mismatched approval; quote and tx replay; concurrent budget exhaustion; duplicate prepare/confirm; wallet rejection / expired blockhash; timeout then successful reconcile; paid delivery retry without duplicate charge; restart persistence + wallet/session isolation.

Reproducible security demo (show real outcomes):

1. Unauthorized merchant request
2. Tampered payment
3. Concurrent requests competing for insufficient remaining budget
4. Legitimate purchase that completes successfully

Label fixtures and injected malicious content.

Defer list

Do **not** add: cross-chain bridges, prediction markets, escrow, new custody vault, marketplace rankings, NFT receipts, NFC, FHE, fiat off-ramps, decorative charts, fake activity, unsupported counters, or security badges.

Delivery order

1. Verified baseline
2. Payment correctness and security
3. Useful agent workflow
4. Clear interface and evidence
5. Optional Streams or sponsorship

Doc status note

This review drives planning. Application code changes are **out of scope for this PDF revision** (“only modify in pdf for now”). Implement against this plan in a later pass.

2. Win plan

Disclaimer: *Simulated judges based on plan + gallery notes — **not** a verified code audit or organizer judging. See JUDGE-REVIEW.md.*

Submit: Fri Nov 20, 12:00 PM CST · **Artifact:** this repo only (`policy-desk`) · **Repo:** private until Devpost flip **Eligibility:** Built during the official hacking window. Do **not** submit prep / `guarded-agent-treasury` as the Devpost project.

Track targets (select on Devpost)

Priority	Track / bounty	Why we fit
Primary	**Agentic Payments and AI**	Research agent buys data under policy + x402 quote→pay→resume
Chain	**Best use of Solana**	Devnet USDC, Phantom sign, TransferChecked settle, explorer receipts
Bounty	**QuickNode Streams** (only if core green + real integration)	Reconcile payments / unmatched transfers — not a decorative ticker
Auto	Official UBC Winner	Clean demo + eligibility + honest craft

Thesis (judge-aligned)

AI agents that can pay are useful until they are dangerous. **Policy Desk** gives a research agent a Solana Devnet USDC budget, enforces allowlists and caps, blocks unsafe purchases, pauses for human approval above threshold, and only delivers paid content after confirmed settlement — with a receipt that ties task → decision → payment → answer.

One-liner for judges: **Agents get a budget — not a blank check.** Differentiation: **Buy useful information, recover from a block, finish the task with proof.**

Agreed product direction

One complete workflow (see 12 steps in JUDGE-REVIEW.md):

A research agent buys useful data within a user-defined budget, handles a blocked purchase safely, requests approval when necessary, and returns an answer supported by purchased information and payment receipts.

Narrow catalog · labeled merchant · fixture mode allowed if visible · no marketplace.

90-second judge story (aligned to workflow)

1. **Start task** — connect Phantom (Devnet); set USDC budget; trust strip shows cluster, mint `4zMMC9…`, wallet, posture.
2. **Block → recover** — agent requests prohibited resource → **blocked** with visible reason → picks approved alternative.
3. **Approve** — above-threshold purchase pauses on SeaPay-style approval card → operator Approve.
4. **Settle → answer** — simulate → Phantom sign → confirm settle → deliver content → agent uses it in final answer → structured receipt.
5. **Close** — “The model proposes. Policy decides. Humans co-sign the big ones. Receipts prove what was bought.” (Streams only if real.)

Killer demo path (one path only)

```
start task + budget → preview catalog → propose resource
  → policy evaluate → BLOCK (recover) | needs_approval | allow
  → operator approve (if needed) → simulate → Phantom sign
  → confirm settlement → deliver paid content → answer + receipt
```

Everything else supports this path. Do not fork into a second product narrative.

What NOT to build (eligibility + focus)

Skip	Why
New on-chain custody vault / Anchor escrow	Judges deferred vault; honesty = **block before sign**
Sponsored/gasless fee payer (as P0)	Optional **after** core; faucet path is enough
Arc, Circle Agent Wallets (no Solana), CCTP bridges	Off-track / time sink
Gemini, ElevenLabs, Grok as core UX	Distracts from Solana + agentic payments
Full Agent Kit plugin surface / agent marketplace	Scope explosion
NFT attestation mints, prediction markets, NFC, FHE, fiat off-ramps	Deferred
Silent “live model” when running fixtures	Label fixture mode; never fake live execution
Submitting prep GAT repo	Eligibility risk — **policy-desk only**

Success criteria

Gate	Done when
Workflow completeness	All 12 demo steps runnable (fixture OK if labeled)
Demo reliability	Cold-start → block + recover + approve + settle + answer + receipt ≤90s, twice
Policy honesty	Non-allowlisted spend never reaches Phantom; rejection reasons visible
Payment correctness	Quote bound to tx; settlement confirmed before content; retries idempotent
Chain truth	Settled rows have Solana explorer links on Devnet USDC mint
Security evidence	Adversarial suite + 4-step security demo with real outcomes
Streams	Claim only if authenticated provider events reconcile payments
Eligibility	Git history starts in hackathon window; Devpost → this repo
Submit package	README + docs, 2–3 min video, Devpost writeup, live tunnel or deploy URL

Engineering team (RACI)

Professionals by discipline — full-stack, cybersecurity, and blockchain — with explicit ownership (no invented employers, years, or certs).

Person	Role	Owns
Dipak Yadav (@dipak5501)	Full-stack developer · product/engineering lead	Task timeline UX, API routes, demo script, Devpost, integration glue
Utkarsh Lubal (@ubiiii)	Cybersecurity engineer	Threat model, operator gate, adversarial tests, audit honesty, security demo
Kashif Manzer (@KashifManzer)	Blockchain engineer	Solana RPC/USDC, intent binding, TransferChecked template validation, settle/confirm, x402, Streams

Review gates: Kashif signs off settle/x402; Utkarsh signs off trust boundaries; Dipak owns merge + judge path.

Delivery order (from judges)

1. Verified baseline
2. Payment correctness and security
3. Useful agent workflow
4. Clear interface and evidence
5. Optional Streams or sponsorship

Day map: TEAM-SPLIT.md. Backlog: FEATURE-BACKLOG.md. Review detail: JUDGE-REVIEW.md.

3. Architecture

Policy-gated Solana Devnet USDC spending for a **research agent** that buys useful data. Settlement is **non-custodial**: Phantom holds keys; the desk decides whether a spend may be prepared, confirmed, and fulfilled.

*Target workflow: task + budget → preview → propose → policy → block/recover or approve → sign → **confirm settle** → deliver content → answer + receipt. Detail: JUDGE-REVIEW.md · WIN-PLAN.md.*

End-to-end research-agent flow



State machine (payment)

`proposed` → `blocked` | `pending_approval` | `approved` | `ready_to_sign` → `confirming` → `settled` → `fulfilled` (plus `rejected` / failed / `unknown` / `paid_delivery_pending` paths). Never show optimistic success for unconfirmed payments.

Signing honesty (five layers)

Layer	Meaning
Policy authorization	Server rules allow this intent
Operator approval	Human gate above threshold (if required)
Wallet signature	Phantom signs prepared bytes
On-chain confirmation	Tx succeeded at documented commitment
Resource delivery	Paid content released; answer may use it

Under-threshold ≠ autonomous settlement if Phantom still signs.

x402 / paid-resource branch

```
resource quote (expiry + exact base units)
  → policy + prepare + Phantom sign
  → confirm settlement (idempotent / reconcile)
  → deliver body / digest
  → resume agent answer
```

CAIP-2 network: `solana:EtWTRABZaYq6iMfeYKouRu166VU2xqa1` (Devnet). Mint: Circle Devnet USDC `4zMMC9srt5Ri5X14GAgXhaHii3GnPAEERYPJgZJDncDU`.

Module map (current + target)

Layer	Path	Responsibility
Desk UI	`app/page.tsx`, `components/*`	Task timeline, catalog preview, approval card, progress, receipts, resource strip
Propose / task	`app/api/agent/*`	Task + resource request → `reviewSpend` → ledger intent
Policy	`lib/policy/*`	Caps, allowlists, mint, pause, approval threshold, regime → real rules
Catalog	*(target)* server-side approved resources	Schema-validated resource IDs; merchants; previews; prices
Approvals	`app/api/approvals/[id]`	Operator-gated approve/reject bound to intent
Settle	`app/api/settle/prepare`, `confirm`	Re-check policy, simulate, template validate, bind prepareId; verify + broadcast + reconcile
Solana	`lib/solana/*`	Connection, balances, TransferChecked build/**full template** validate
x402 / delivery	`lib/x402/*`, resource routes	Quote, seller verify, settle-before-deliver, retry fulfillment without re-charge
Ledger / budget	`lib/ledger/store`	Durable `.data/ledger.json`; atomic reservations; concurrency model documented
Audit	`lib/security/audit`	Append-only `.data/audit.jsonl` (server-controlled — not on-chain immutability)
Streams	`app/api/streams/webhook`	Optional reconcile / unmatched-transfer flags (not ticker theater)
Operator	`lib/security/operator`	Shared-secret gate (`OPERATOR_TOKEN`); never exposed to agent

Trust boundaries

```
[Untrusted]
  Browser UI · model/agent text · seller HTTP · Streams webhook JSON · fixture scripts
        ■
        ▼
[Trusted desk server]
  catalog resolve · reviewSpend · intent bind · prepare re-check · template validate
  confirm / reconcile · delivery gate · Operator token for approve/reject
        ■
        ▼
[Wallet boundary]
  Phantom signs only the prepared transaction bytes
  Desk never holds user keys
        ■
        ▼
[Solana Devnet]
  SPL USDC TransferChecked / x402 exact settle
  Explorer = settlement truth for receipts
```

Boundary	Rule
Model → policy	Agent text never sets caps, mint, allowlists, operator creds, or arbitrary RPC/merchant URLs
Browser → approvals	`x-operator-token` / Bearer must match `OPERATOR_TOKEN` (fail closed if unset)
Browser → settle	Prepare re-runs policy + destination binding; confirm validates **full** permitted template
Seller → desk	Exact amount, mint, payTo, signed transfer shape; delivery only after confirm
Webhook → ledger	Streams verifies provider auth when configured; dedupe; else do not claim bounty
Config → cluster	Devnet lock — refuse mainnet RPC/cluster configuration

Security foundation (architecture requirements)

1. **Server policy** — only path to materialize a payment.
2. **Intent binding** — wallet, task, resource, merchant, network, mint, base-unit amount, quote expiry, policy version.
3. **Integer base units** — no float money math.
4. **Atomic budgets** — concurrent requests cannot exceed remaining; release once; uncertain stays reserved until reconciled.
5. **Template validation** — entire tx vs allowlisted instruction set (not “find one TransferChecked”).
6. **Idempotent retries** — reconcile before replace; timeout ≠ failure.
7. **Settle before fulfill** — delivery retries do not re-charge.
8. **No vault this build** — block-before-sign; sponsorship optional later.

Devnet USDC

Item	Value
Cluster	`devnet` (`NEXT_PUBLIC_SOLANA_CLUSTER`)
RPC	`SOLANA_RPC_URL` (prefer private QuickNode; fallback public Devnet)
Mint	`4zMMC9srt5Ri5X14GAgXhaHii3GnPAEERYPJgZJDncDU`
Decimals	6 (base units: `1_000_000` = $1)
Wallet	Phantom · user switches network to Devnet
Faucets	SOL: faucet.solana.com · USDC: faucet.circle.com

Design choices (production-minded prototype)

1. **Block before sign** — vault deferred; honesty over custody theater.
2. **One research workflow** — small catalog beats marketplace.
3. **Fixture mode labeled** — reliable demos without faking live models.
4. **Double policy check** — propose and prepare both call `reviewSpend`.
5. **Audit honesty** — local append-only ≠ independently immutable on-chain trail.
6. **Single-process ledger** — fine for hackathon demo; concurrency model must still be defined and tested.

Related docs

- JUDGE-REVIEW.md — simulated panel decisions
- WIN-PLAN.md — demo path and non-goals
- FEATURE-BACKLOG.md — P0 rewritten to judges
- TEAM-SPLIT.md — 5-day execution
- THREAT-MODEL.md — guarantees and limitations
- STREAMS.md — optional QuickNode wiring
- ORGANIZER-RESOURCES.md — sponsor links and status

4. Feature backlog (P0)

Priorities rewritten to match the **three-judge agreed decision** (JUDGE-REVIEW.md): one complete research-agent workflow, payment correctness, and security evidence first.

Gallery inspiration from MBC 2025 notes (`guarded-agent-treasury/docs/MBC-2025-GALLERY-NOTES.md`). Treat notes as inspiration — do not repeat unverified award claims or copy another project's code/branding.

P0 = demo-critical · **P1** = after P0 green · **P2** = optional / deferred.

Owners: **Dipak** (full-stack / product) · **Utkarsh** (cybersecurity) · **Kashif** (blockchain).

P0 — research workflow + payment correctness + security evidence

ID	Feature	Gallery / judge source	Acceptance criteria	Owner	Status
P0-1	Task + budget start ("Run a task")	Judge workflow §1	User starts research task with defined USDC budget; timeline is central	Dipak	**Plan** — align UI to task-first
P0-2	Resource catalog preview	Quarry	Free preview, description, price, merchant visible before purchase	Dipak · Kashif catalog	**Plan**
P0-3	Policy evaluate via server API	Agent Guard	Resource request → deterministic rules; model cannot set recipients/mint/limits	Utkarsh · Dipak	**In app (partial)** — extend to resource IDs
P0-4	Block + recover path	Product judge	Prohibited request blocked with concrete reason; agent chooses approved alternative	Dipak · Utkarsh	**In app (block)** — recover narrative **Plan**
P0-5	Approval card in task timeline	SeaPay	Above-threshold pauses; shows merchant, resource, amount, reason, budget impact, expiry; progress survives refresh	Dipak · gate Utkarsh	**In app** — harden into timeline
P0-6	Intent binding + quote expiry	Blockchain judge	Intent bound to wallet, task, resource, merchant, network, mint, base-unit amount, quote expiry, policy version	Kashif · Utkarsh	**Partial** — harden
P0-7	TransferChecked **template** validation	Security / chain	Full tx matches permitted instruction template; reject extras (delegates, authority changes, unexpected programs, etc.)	Kashif · Utkarsh	**Partial** — "one TransferChecked" insufficient
P0-8	Atomic budget reservation	FRCT + security	Available / reserved / confirmed / remaining visible; concurrent requests cannot overspend; release once on reject/expiry	Kashif · Dipak	**Partial**
P0-9	Idempotent prepare/confirm + reconcile	Blockchain judge	Retry reconciles existing tx before replacement; timeout ≠ failure; settle only after network confirm at documented commitment	Kashif	**Plan / harden**
P0-10	Settlement before content delivery	Quarry	Paid content only after confirmed settle; delivery retry without re-charge	Kashif · Dipak	**Plan**
P0-11	Answer uses purchased content	Product judge	Final answer cites/uses delivered resource; fixture mode labeled if used	Dipak	**Plan**
P0-12	Structured receipt	Quarry / PolyHedge	Receipt links task, intent, quote, policy version, decision, approval ref, sig, network, mint, amount, merchant, confirm status, content digest; export without NFT	Dipak · Kashif	**Partial** — expand fields
P0-13	Visible policy pass/fail reasons	Agent Guard	Desk shows which rules passed/failed; distinguish model justification from policy evaluation	Dipak · Utkarsh	**In app** — strengthen copy
P0-14	Adversarial tests + security demo	Security judge	Suite covers unauthorized, tamper, concurrent budget, happy path; 4-step demo with real outcomes	Utkarsh · Kashif	**Partial** — expand to checklist

ID	Feature	Gallery / judge source	Acceptance criteria	Owner	Status
P0-15	Dry-run / simulate before broadcast	Agent Guard, PolyHedge	Insufficient SOL/USDC surfaces before sign; dry-run does not spend	Kashif · Dipak	**In app**
P0-16	Trust strip + organizer resources	Density/trust	Devnet, mint `4zMMC9…`, wallet, posture + ResourceStrip links	Dipak	**In app**
P0-17	90s demo reliability	Win plan	Full block→recover→approve→settle→answer→receipt twice cold	All	**Active**

P1 — high leverage after P0

ID	Feature	Source	Acceptance criteria	Owner
P1-1	Fee / balance clarity	Gasless Hub lesson / FRCT	SOL + USDC; clear insufficient errors	Dipak · Kashif
P1-2	Posture presets map to enforced rules	FRCT	Defensive/Neutral/Aggressive (or equiv) = real policy, not labels only	Dipak · Utkarsh
P1-3	Private Devnet RPC	Reliability	QuickNode (or equiv) in `.env.local`	Kashif
P1-4	Public tunnel / deploy URL	Judging	Stable URL for video + Devpost	Dipak
P1-5	Pitch video 2–3 min	Submit	Problem → workflow → live clip → threat honesty → close	Dipak · all

P2 — optional / deferred (judges)

ID	Feature	Source	Decision
P2-1	Sponsored / gasless fees	Gasless Hub	**Optional after core** — fee limits, abuse controls, ATA cost disclosure
P2-2	QuickNode Streams reconcile	Streams bounty	**Conditional** — real provider events only; else drop claim
P2-3	On-chain custody vault / escrow	Aegis-like	**Deferred**
P2-4	NFT / attestation receipt mint	PolyHedge, Quarry	Deferred — structured export is enough
P2-5	Chat-first marketplace / rankings	SeaPay, agent rank	Out of scope
P2-6	CCTP / bridges / prediction / NFC / FHE / fiat	Various	Deferred

Definition of “judge P0 done”

Without coaching beyond the 90s script, judges can see:

1. **Task + budget** started.
2. **Preview** of a purchasable resource.
3. A **block** with concrete reason and **recovery** to an approved resource.
4. An **approval** interrupt, then Approve.
5. **Simulate** → **sign** → **confirm settle** → **deliver content** → **answer**.
6. A **receipt** tying task / payment / resource / answer.
7. A **security demo** (unauthorized, tamper, concurrent budget, success) with real outcomes.

Anything that does not serve that sequence is not P0.

5. Team split (5-day)

Three working engineers shipping the **one complete research-agent workflow** agreed by simulated judges (JUDGE-REVIEW.md). Clear RACI, parallel streams, daily DoD.

Delivery order: baseline → payment correctness → agent workflow → UI/evidence → optional Streams.

Roster

Engineer	GitHub	Discipline	Accountable for
Dipak Yadav	@dipak5501	Full-stack developer · product/engineering lead	Task timeline UX, catalog/preview UI, approval card, receipt/answer surface, demo script, Devpost, merge cadence
Utkarsh Lubal	@ubiiii	Cybersecurity engineer	Threat model, operator auth, adversarial suite, security demo evidence, audit honesty, policy claim accuracy
Kashif Manzer	@KashifManzer	Blockchain engineer	Intent binding, TransferChecked template validation, atomic budgets, idempotent settle/confirm, x402 delivery gate, Streams (if claimed)

RACI: Dipak = **A** on product demo · Kashif = **A** on chain settle · Utkarsh = **A** on security claims · each is **C** on the others' P0 merges.

Day	Date (2026)	Theme
1	Sat Nov 15	Verified baseline + gap list vs 12 steps
2	Sun Nov 16	Payment correctness + security foundation
3	Mon Nov 17	Useful agent workflow (block→recover→approve→answer)
4	Tue Nov 18	UI/evidence polish; Streams only if core green
5	Wed Nov 19 → Thu Nov 20 AM	Pitch, video, Devpost; submit by noon CST Fri

(If the team works Fri Nov 14, treat it as Day 1 early start; do not slip Day 5.)

Day 1 — Verified baseline

Parallel

Stream	Owner	Work
Full-stack	Dipak	Map current UI to 12 demo steps; cold-run existing block/approve/settle; list gaps; keep trust strip accurate
Blockchain	Kashif	Verify Devnet mint, balances, prepare simulate, confirm TransferChecked happy path; note template-validation gaps
Security	Utkarsh	Baseline adversarial run; expand THREAT-MODEL.md honesty notes; operator fail-closed check

DoD

- [ ] Written baseline: verified / partial / mocked / documented-missing (no invented results).
- [ ] Gap list mapped to P0 IDs in FEATURE-BACKLOG.md.
- [ ] `npm test` + `npm run build` green on current main.

Day 2 — Payment correctness + security

Parallel

Stream	Owner	Work
Blockchain	Kashif	Intent binding fields; full instruction-template validation; atomic reservation concurrency; idempotent retry/reconcile; settle-before-delivery gate
Security	Utkarsh	Tests: wrong mint/amount/recipient/network; extra instructions; approval mismatch; replay; concurrent budget; duplicate prepare/confirm
Full-stack	Dipak	Surface reservation/budget states; never optimistic “paid” before confirm; error states for reject/timeout/unknown

DoD

- [ ] Concurrent requests cannot exceed remaining budget.
- [ ] Malicious extra-instruction txs rejected.
- [ ] Timeout path reconciles before re-prepare.
- [ ] Security demo steps 1–3 (unauthorized, tamper, concurrent) show real outcomes.

Day 3 — Useful agent workflow

Parallel

Stream	Owner	Work
Full-stack	Dipak	Task-first timeline; catalog preview; block→recover UX; approval card in timeline; answer + receipt after delivery
Blockchain	Kashif	Small approved resource catalog + merchant wallet; quote expiry; x402/resource delivery after confirm
Security	Utkarsh	Visible rejection reasons; fixture vs live labeling; operator credentials never exposed to agent path

DoD

- [ ] Steps 1–12 runnable (fixture mode OK if labeled).
- [ ] Blocked request never materializes a signable tx.
- [ ] Above-threshold requires operator Approve before Phantom.
- [ ] Final answer uses purchased content; receipt fields filled.

Day 4 — Interface, evidence, optional Streams

Parallel

Stream	Owner	Work
Full-stack	Dipak	Expandable inspection views; loading/empty/approval/signing/rejected/failed/confirming/unknown/paid-pending/completed; backup recording
Blockchain	Kashif	Streams **only if** core green: auth, dedupe, reconcile known payments — else formally drop bounty claim
Security	Utkarsh	Finalize 4-step security demo; audit-honesty language; webhook secret review if Streams claimed

DoD

- [ ] Security demo step 4 (legitimate success) + full adversarial suite green.
- [ ] Streams live end-to-end **or** bounty deselected everywhere.
- [ ] Full 90s path recorded as backup.

Day 5 — Ship package (Wed Nov 19 → Fri Nov 20 noon CST)

Parallel

Stream	Owner	Work
Lead	Dipak	Devpost writeup, video edit, repo public flip, final README, live URL
Blockchain	Kashif	Pre-flight faucet balances; RPC health; settle smoke 30 min before submit
Security	Utkarsh	Final threat-model quote for Devpost; rotate demo operator token if URL is public

Submit checklist

- [ ] Devpost artifact = **policy-desk only** (not prep GAT).
- [ ] Tracks: Agentic Payments and AI + Best use of Solana (+ Streams only if live).
- [ ] Video ≤3 min: task → block/recover → approve → settle → answer → receipt.
- [ ] Repo public (or per organizer rule) at submit time.
- [ ] Hard stop: **Fri Nov 20, 12:00 PM CST**.

What each person does next (immediate)

Engineer	Next actions
Dipak	Own task-timeline UX plan from 12 steps; keep docs ↔ product aligned; schedule Devpost draft
Utkarsh	Own adversarial expansion + security demo script; audit honesty in pitch language
Kashif	Own payment correctness (template validation, budgets, idempotent settle); Streams decision after core

Communication

- Merge through Dipak unless hotfix on chain/security.
- No scope adds without cutting something from FEATURE-BACKLOG.md P0.
- Overclaim ban: if it is not enforced in code + tests, it is not in the pitch.
- This doc revision is planning-only until the implementation pass.

6. Threat model

Owner: Utkarsh Lubal (@ubiiii), cybersecurity engineer **Reviewers:** Dipak Yadav (product/engineering), Kashif Manzer (blockchain) **Scope:** Solana **Devnet** hackathon prototype (UBC 2026). Not a production custody product. **Audit status:** No third-party security audit has been performed on this repository.

Assets

Asset	Why it matters
USDC in the connected Phantom wallet	Devnet funds the human controls; desk must not trick-sign drains
Daily / per-tx spend budget	Prevents agent overspend within the desk session
Allowlisted payees & resources	Stops arbitrary merchant drains
Ledger + audit trail	Demo integrity and post-hoc review
Operator token	Gates human approve / reject
Streams webhook secret	Prevents forged LIVE feed events when bounty path is enabled

Trust boundaries

```
[Untrusted] Browser UI, agent/scenario text, seller HTTP, webhook JSON
      ■
      ▼
[Server policy + ledger]  reviewSpend → immutable intent → prepare/confirm
      ■
      ▼
[Wallet] Phantom signs only what prepare built
      ■
      ▼
[Solana Devnet] SPL USDC / x402 seller self-facilitate
```

Boundary	Rule
Model → policy	Scenario/agent text never sets caps, mint, or allowlists
Browser → approvals	`x-operator-token` / Bearer must match `OPERATOR_TOKEN` (fail closed if unset)
Browser → settle	Prepare re-checks policy + destination; confirm validates TransferChecked
Seller → desk	x402 seller verifies amount, mint, payTo, and signed transfer shape
Webhook → ledger	Streams verifies shared secret **when configured**; otherwise acknowledge-only
Config → network	Cluster/RPC guards refuse mainnet configuration

Enforced guarantees (this build)

1. **Policy before materialization** — `reviewSpend` on propose and again on prepare; blocked spends never get a transaction.
2. **Immutable payment intent** — `payee`, `amount`, `mint`, and `resource` cannot be patched after create (`intentHash`).
3. **Destination binding at settle** — payee must be allowlisted **or** equal the spending wallet on an allowlisted resource (pay-self demo).
4. **Signed tx matches intent** — confirm decodes TransferChecked (mint, amount, ATAs, authority) before treating settle as valid; signature-only confirm is rejected.
5. **Human gate above threshold** — `needs_approval` requires operator Approve before prepare/sign.
6. **Budget reservation** — pending / approved / ready_to_sign / confirming / settled count toward the daily cap.
7. **Replay resistance (desk-level)** — settled entries cannot be settled again; prepare binds `prepareId` + expiry.
8. **Devnet lock** — refuse mainnet cluster/RPC configuration.
9. **Audit log** — status transitions append to durable audit trail (best-effort `.data/audit.jsonl`).

Honest limitations

Limitation	Why
Operator token is a shared secret	Demo auth — not SSO, WebAuthn, or wallet-signed approvals

Limitation	Why
In-process / file ledger	Not multi-region HA; wiping `.data` loses history; not a bank ledger
No on-chain policy program	Caps live in the Next.js process, not an Anchor vault
Safety model is block-before-sign	No escrow PDA locks funds; honesty over custody theater
x402 seller is a local demo facilitator	Not a hosted facilitator / compliance network
Streams webhook	Secret must be set for untrusted internet exposure; optional bounty path
Phantom UX	User can still sign unrelated txs outside this desk
Public tunnel / deploy URL	Anyone with the URL + operator token can hit approval APIs — rotate token for public demos
RPC / faucet availability	External dependency; dry-run still shows policy path if chain is down

Adversarial coverage

Primary suites: `lib/security/adversarial.test.ts`, policy review tests, TransferChecked validation tests.

Expected red-team cases (owners: Utkarsh design · Kashif chain fixes · Dipak UI honesty):

- Signature-only confirm rejection
- Wrong-mint / over-cap / non-allowlisted propose blocks
- Destination allowlist enforcement
- TransferChecked amount/payee mismatch detection
- Approval without valid operator token rejected

What we will not claim in pitch

- Production-grade custody, MPC, or hardware-enforced agent keys
- Formal verification or external audit
- Mainnet readiness
- That policy lives on-chain in this MVP

Related

- ARCHITECTURE.md — flow and boundaries
- FEATURE-BACKLOG.md — P1 adversarial expansion
- STREAMS.md — webhook secret configuration

7. Organizer resources

Real UBC Hackathon 2026 / sponsor resources only. **No invented APIs.** Owners: Dipak (product wiring) · Kashif (chain/RPC/Streams) · Utkarsh (webhook secret / abuse review).

Resource	Link	Wired in policy-desk	Owner	Status
Solana Devnet RPC	api.devnet.solana.com / private QuickNode HTTP URL via `SOLANA_RPC_URL`	`lib/solana/connection.ts`, `.env.example`	Kashif	**Wired** — prefer private endpoint for demos
Circle Devnet USDC mint	USDC contract addresses · `4zMMC9srt5Ri5X14GAgXhaHii3GnPAEERYPJgZJDncDU`	Policy allowlist, balances, settle, trust strip	Kashif	**Wired**
Phantom wallet	phantom.app/download	Connect + sign (`WalletProvider`, settle path)	Dipak + Kashif	**Wired** — judges must switch to Devnet
x402 on Solana (V2)	Agentic payments / x402 · CAIP-2 `solana:EtWTRABZaYq6iMfeYKouRu166VU2xqa1`	`/api/x402/weather` quote → pay → resume	Kashif	**Wired**
SOL faucet	faucet.solana.com	`ResourceStrip` + README	Dipak	**Linked**
Circle USDC faucet	faucet.circle.com · Solana / Devnet / USDC	`ResourceStrip` + README	Dipak	**Linked**
QuickNode Streams	Streams quickstart · Solana program data guide	`POST/GET /api/streams/webhook` + LIVE ticker	Kashif · secret review Utkarsh	**Wired (optional bounty)** — configure stream + `QUICKNODE_STREAM_WEBHOOK_SECRET`; see STREAMS.md
Circle docs (stablecoins)	developers.circle.com	Mint constant + UI note	Kashif	**Referenced**
Solana explorer (Devnet)	Receipt links from settle confirm	Settled ledger / UI receipts	Kashif	**Wired**

UI surface

`components/ResourceStrip.tsx` exposes faucet, Phantom, x402, mint, and Streams links on the desk.

SKIP (per track / scope)

Skip	Reason
Arc	Not Solana demo path
Circle Agent Wallets	No Solana
CCTP cross-chain	Out of killer demo path
Gemini / ElevenLabs / Grok as core	Not needed for Solana + agentic payments story
Full Agent Kit plugin matrix	Scope

Pre-judge faucet checklist

1. Phantom → **Devnet**

2. SOL ≥ ~0.02 from faucet.solana.com
3. USDC ≥ ~0.60 from faucet.circle.com (Solana / Devnet / USDC)
4. Confirm trust strip mint matches `4zMMC9srt5Ri5X14GAgXhaHii3GnPAEERYPJgZJDncDU`
5. If claiming Streams bounty: webhook secret set + one LIVE `stream` event visible (STREAMS.md)

Companion docs

- Compact status table (legacy): RESOURCES.md
- Streams setup: STREAMS.md

7b. Resources (short index)

Full integration checklist with owners + status: **ORGANIZER-RESOURCES.md**. **Streams bounty setup:** STREAMS.md**. **Win strategy:** WIN-PLAN.md**.

Quick links (official)

Need	URL
SOL faucet	https://faucet.solana.com
USDC faucet (Circle)	https://faucet.circle.com/
Phantom	https://phantom.app/download
x402 on Solana	https://solana.com/docs/payments/agentic-payments/x402
Circle USDC addresses	https://developers.circle.com/stablecoins/usdc-contract-addresses
QuickNode Streams	https://www.quicknode.com/docs/streams/quickstart
UBC Devpost	https://ubc-hackathon-2026.devpost.com/

Devnet USDC mint: `4zMMC9srt5Ri5X14GAgXhaHii3GnPAEERYPJgZJDncDU` **x402 CAIP-2:** `solana:EtWTRABZaYq6iMfeYKouRu166VU2xqa1`

7c. QuickNode Streams

Streams feeds Solana Devnet USDC activity into Policy Desk so judges see an **on-chain audit trail** next to policy decisions.

What shipped

1. `POST /api/streams/webhook` — verifies `QUICKNODE_STREAM_WEBHOOK_SECRET`, appends events to the durable ledger feed.
2. `GET /api/streams/webhook` — returns recent stream events for the LIVE ticker.
3. Desk **LIVE** ticker shows `stream` hits alongside blocked / settled ledger rows.

Configure

1. Create a QuickNode account + **Solana Devnet** endpoint (also set `SOLANA_RPC_URL` to that HTTP URL).
2. Create a Stream filtered to Token Program transfers involving:
- Devnet USDC mint `4zMMC9srt5Ri5X14GAgXhaHii3GnPAEERYPJgZJDncDU`
- Your treasury wallet and/or demo payee
3. Destination: HTTPS webhook → `https://<your-tunnel-or-deploy>/api/streams/webhook`
4. Set a shared secret:

```
QUICKNODE_STREAM_WEBHOOK_SECRET=replace-me
# Accepted headers: x-qn-signature | x-webhook-secret | Authorization: Bearer …
```

Field	Value
Stream id	_(fill after create)_
Network	Solana Devnet
Filter	USDC mint + treasury/payee ATAs
Webhook	`/api/streams/webhook`

Local test without QuickNode

```
curl -X POST http://localhost:3000/api/streams/webhook \
  -H 'Content-Type: application/json' \
  -H 'x-webhook-secret: replace-me' \
  -d '{"signature":"DemoSig1111111111111111111111111111111111111111","note":"manual"}'
```

Refresh the desk — the LIVE ticker should show a `stream` item.

Links (organizer / sponsor)

- Streams quickstart
- Stream Solana program data

Appendix A — Organizer & sponsor links

Every organizer/sponsor URL in one place. Prefer the clickable link; full URL is printed.

Resource	URL
Solana Devnet RPC	https://api.devnet.solana.com
Circle USDC contract addresses	https://developers.circle.com/stablecoins/usdc-contract-addresses
Phantom download	https://phantom.app/download
x402 on Solana (agentic payments)	https://solana.com/docs/payments/agentic-payments/x402
SOL faucet	https://faucet.solana.com
Circle USDC faucet	https://faucet.circle.com/
QuickNode Streams quickstart	https://www.quicknode.com/docs/streams/quickstart
QuickNode: stream Solana program data	https://www.quicknode.com/guides/quicknode-products/streams/how-to-stream-solana-program-data
Circle developers (stablecoins)	https://developers.circle.com/stablecoins/usdc-contract-addresses
UBC Hackathon 2026 Devpost	https://ubc-hackathon-2026.devpost.com/
Solana Explorer (Devnet)	https://explorer.solana.com/?cluster=devnet
Solana Explorer tx template	https://explorer.solana.com/tx/<SIGNATURE>?cluster=devnet
Aegis (conceptual competitor — Devpost)	https://devpost.com/software/aegis-8eg7pj
Agent Guard (conceptual competitor — Devpost)	https://devpost.com/software/agent-guard-hvg45j
Dipak Yadav GitHub	https://github.com/dipak5501
Utkarsh Lubal GitHub	https://github.com/ubiiii
Kashif Manzer GitHub	https://github.com/KashifManzer
policy-desk private repo	https://github.com/dipak5501/policy-desk

Appendix B — Constants

Constant	Value
Devnet USDC mint	`4zMMC9srt5Ri5X14GAgXhaHii3GnPAEERYPJgZJDncDU`
x402 CAIP-2 (Devnet)	`solana:EtWTRABZaYq6iMfeYKouRu166VU2xqa1`
Decimals	6 (1_000_000 base units = $1)
Hard submit	Fri Nov 20, 2026 · 12:00 PM CST

End of planning PDF. Implement against JUDGE-REVIEW / FEATURE-BACKLOG in a later code pass — this revision intentionally changes documentation only.